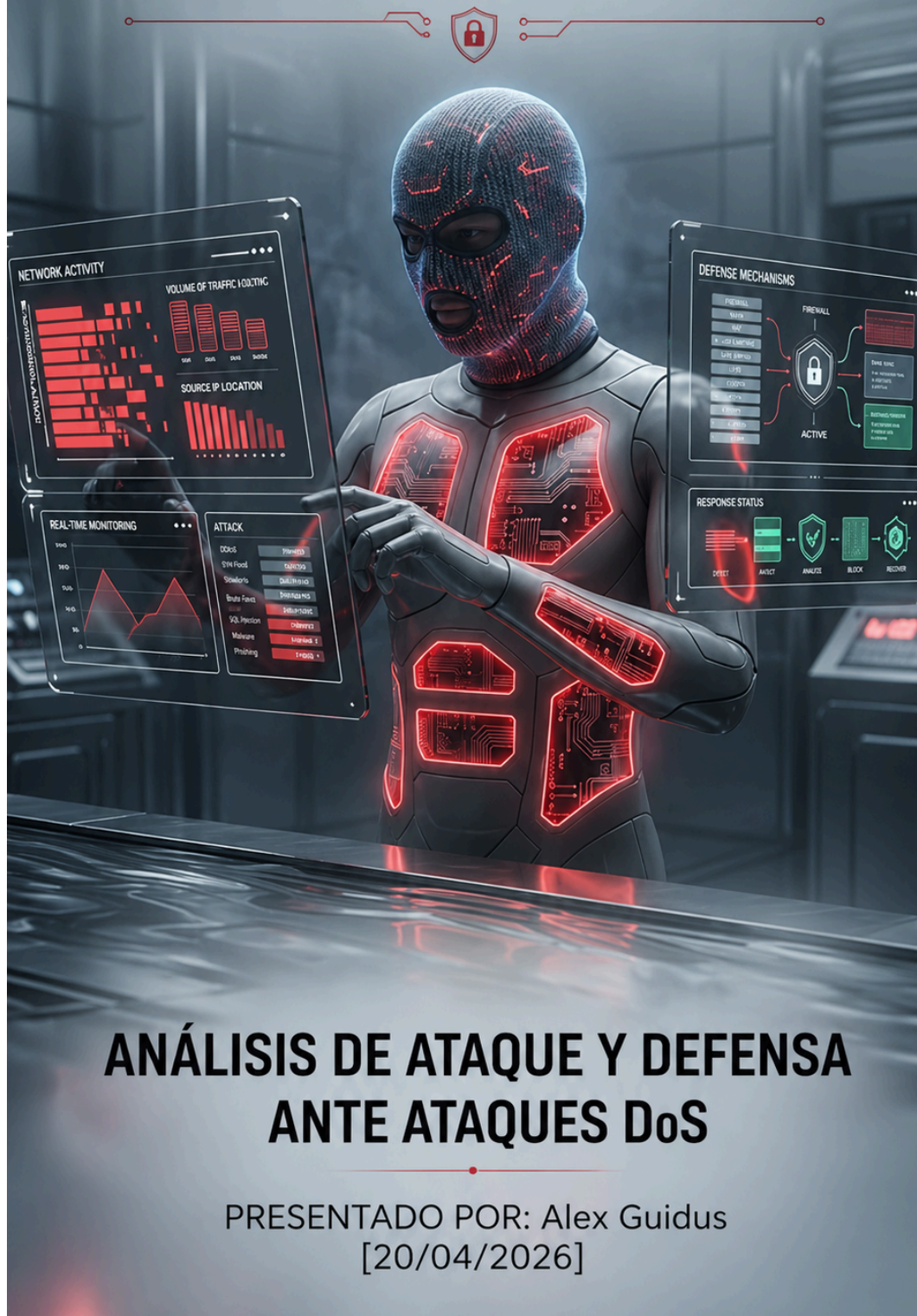


TRABAJO FINAL DE CIBERSEGURIDAD



ANÁLISIS DE ATAQUE Y DEFENSA ANTE ATAQUES DoS

PRESENTADO POR: Alex Guidus
[20/04/2026]

Proyecto Final – Ataque y Defensa ante Ataques DoS

Índice

1. Introducción.....	3
2. Arquitectura del entorno.....	4
3. Configuración del sistema.....	5
4. Despliegue del servicio web.....	6
5. Línea base y monitorización del sistema.....	7
6. Ejecución de los ataques Slowloris y SYN Flood.....	8
7. Implementación de medidas de mitigación.....	18
8. Validación de las medidas de mitigación.....	20
9. Conclusiones finales.....	22
10. Anexos.....	24

1. Introducción

En el contexto actual de la ciberseguridad, la disponibilidad de los servicios expuestos a red es un aspecto crítico. Los ataques de denegación de servicio (DoS) representan una de las amenazas más comunes, ya que buscan impedir el acceso legítimo a un sistema mediante la saturación de sus recursos.

El presente proyecto tiene como objetivo analizar el comportamiento de un servidor web ante distintos tipos de ataques DoS, así como evaluar la efectividad de diferentes medidas de mitigación. Para ello, se ha diseñado un entorno de laboratorio controlado basado en máquinas virtuales, donde se simulan escenarios realistas de ataque y defensa sin comprometer sistemas reales.

En este entorno, se ha desplegado un servidor Ubuntu con Apache que aloja una aplicación web sencilla, junto con una máquina Kali Linux que actúa como origen de los ataques. A lo largo del proyecto se han ejecutado dos tipos principales de ataques: Slowloris (capa de aplicación) y SYN Flood (capa de transporte), permitiendo observar cómo afectan a diferentes niveles del sistema.

Es importante destacar que la intención inicial del proyecto era simular un ataque distribuido (DDoS) utilizando varias máquinas dentro del entorno del laboratorio. Sin embargo, debido a las limitaciones de recursos del equipo anfitrión, no fue posible mantener múltiples sistemas atacantes funcionando de forma simultánea sin comprometer el rendimiento general. Por este motivo, se optó por adaptar el escenario a un ataque de tipo DoS centralizado, utilizando una única máquina Kali Linux como origen de los ataques.

A partir de este análisis, se han implementado distintas medidas de defensa tanto a nivel de aplicación como de sistema, con el objetivo de reducir el impacto de los ataques y mantener la disponibilidad del servicio. Finalmente, se comparan los resultados obtenidos antes y después de aplicar estas medidas, permitiendo extraer conclusiones sobre su efectividad.

En este sentido, el escenario puede relacionarse con **OWASP Top 10 - Security Misconfiguration**, ya que una configuración inadecuada del servidor puede permitir que un atacante abuse de recursos como conexiones concurrentes, timeouts o colas TCP. Las medidas aplicadas, como **mod_reqtimeout**, ajustes en **sysctl** y reglas de filtrado mediante **iptables**, buscan reducir precisamente este riesgo, reforzando la disponibilidad del servicio, dentro de la tríada CIA (Confidencialidad, Integridad y Disponibilidad).

Este proyecto no solo busca demostrar el impacto de los ataques DoS, sino también entender su funcionamiento y aprender a mitigarlos de forma práctica en un entorno realista.

2. Arquitectura del entorno

2.1 Entorno de virtualización

Para la implementación del laboratorio se ha utilizado el software de virtualización **Oracle VM VirtualBox**, en su versión 7.2.2. Este entorno permite simular un escenario realista de ataque y defensa de forma controlada, sin afectar a sistemas externos.

El laboratorio se ejecuta sobre una máquina anfitriona con las siguientes características:

- Sistema operativo: Windows 11 Pro 25H2
- Memoria RAM: 16 GB
- Procesador: AMD Ryzen 5 5600X

Estas especificaciones han permitido ejecutar varias máquinas virtuales de forma simultánea.

2.2 Máquinas del entorno

El entorno de laboratorio está compuesto por dos máquinas virtuales principales:

Ubuntu Server (Servidor objetivo)

- Sistema operativo: Ubuntu Server 24.04.4 LTS
- Rol:
 - Servidor web (Apache)
 - Alojamiento de la aplicación web
 - Gestión de la base de datos (formato JSON)
- Función principal:
Actúa como sistema objetivo de los ataques, permitiendo analizar su comportamiento ante situaciones de carga y denegación de servicio.

Kali Linux (Máquina atacante)

- Sistema operativo: Kali Linux 2026.1
- Rol:
 - Ejecución de ataques de denegación de servicio
- Herramientas utilizadas:
 - hping3 (SYN Flood)
 - slowloris (ataque a nivel de aplicación)

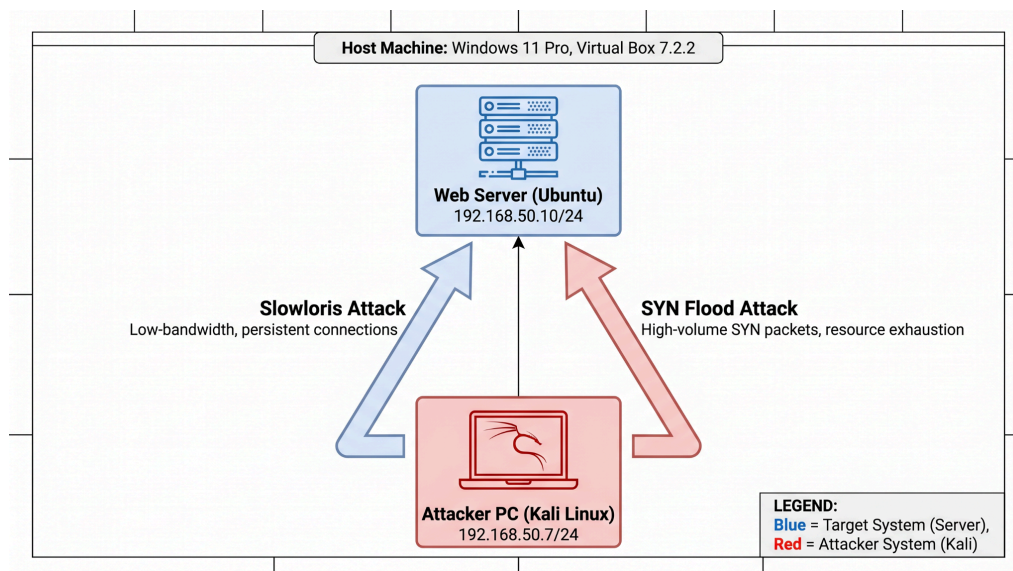
- Función principal:
Generar tráfico malicioso controlado contra el servidor con el objetivo de evaluar su resistencia y analizar el impacto de los ataques.

2.3 Diseño del entorno

Las máquinas virtuales están conectadas mediante una red interna, lo que permite:

- Comunicación directa entre sistemas
- Aislamiento del entorno respecto a redes externas
- Control total sobre el tráfico generado

Este diseño permite simular un escenario realista de ataque sin comprometer la seguridad del sistema anfitrión ni de otros equipos.



La siguiente figura representa la topología del laboratorio, donde un servidor Ubuntu actúa como objetivo de múltiples ataques DoS desde un atacante Kali dentro de una red aislada (192.168.50.0/24).

3. Configuración del sistema

Para la preparación del entorno, se realizó una configuración básica del sistema en el servidor Ubuntu con el objetivo de garantizar estabilidad, conectividad y correcto funcionamiento de los servicios desplegados.

En primer lugar, se estableció una dirección IP estática dentro del rango de la red del laboratorio, asegurando una comunicación consistente entre las máquinas virtuales. Esta configuración resulta fundamental en entornos controlados, donde los cambios dinámicos de IP pueden afectar al acceso a servicios y a la ejecución de pruebas.

Asimismo, se habilitó el acceso remoto mediante SSH, facilitando la administración del servidor desde el sistema anfitrión y permitiendo una gestión más eficiente durante las fases de ataque y monitorización.

*Los comandos utilizados para la instalación y despliegue del servicio web, así como su verificación, se incluyen en el **Anexo A**.*

Por último, se realizaron ajustes básicos del sistema para asegurar la compatibilidad con los servicios utilizados, así como la correcta ejecución del servidor web y las herramientas de análisis.

4. Despliegue del servicio web

4.1 Objetivo del servicio web

Con el fin de disponer de un objetivo realista para las pruebas de disponibilidad, se desplegó un servicio web sencillo sobre el servidor Ubuntu. Este servicio no se planteó como una aplicación compleja, sino como un entorno funcional suficiente para simular un escenario real de acceso a contenidos mediante HTTP.

La aplicación desarrollada muestra una interfaz web básica y carga información desde un fichero JSON que actúa como base de datos ficticia de clientes. De este modo, el servidor no solo responde a peticiones sobre una página estática, sino también a solicitudes relacionadas con la carga de datos, lo que permite observar con mayor claridad el impacto de los ataques sobre el servicio.

4.2 Estructura de la aplicación

La aplicación se organizó en una estructura simple compuesta por los siguientes elementos:

- **index.html**: interfaz principal de la aplicación
- **style.css**: estilos visuales
- **app.js**: lógica de carga y visualización de datos
- **clientes.json**: base de datos ficticia utilizada por la aplicación

La base de datos contiene registros simulados de clientes, incluyendo campos como nombre, correo electrónico, empresa y estado. Todos los datos utilizados son ficticios y se emplean exclusivamente con fines académicos dentro del laboratorio.

4.3 Publicación en el servidor Apache

Para la publicación del servicio web, se instaló y configuró el servidor Apache en el sistema Ubuntu. Posteriormente, los archivos de la aplicación fueron copiados al directorio raíz del servidor web, permitiendo su acceso desde el resto de máquinas del laboratorio. Tras ajustar los permisos correspondientes y reiniciar el servicio, se verificó el correcto funcionamiento de la aplicación mediante peticiones HTTP desde la máquina atacante.

*Los comandos utilizados para la instalación y despliegue del servicio web, así como su verificación, se incluyen en el **Anexo B**.*

5. Línea base y monitorización del sistema

5.1 Despliegue de herramientas de monitorización

Antes de ejecutar cualquier tipo de ataque, se procedió a desplegar un conjunto de herramientas de monitorización en el servidor con el objetivo de observar el comportamiento del sistema en condiciones normales de funcionamiento. Esta fase es fundamental, ya que permite establecer una línea base (baseline) que servirá posteriormente como referencia para comparar el impacto de los ataques.

Para ello, se abrieron múltiples sesiones simultáneas por SSH hacia el servidor, cada una dedicada a una herramienta específica de análisis:

- **htop**: monitorización de uso de CPU, memoria y procesos activos
- **ss -s**: análisis del estado de las conexiones TCP
- **iftop**: monitorización del tráfico de red en tiempo real
- **curl**: comprobación de la disponibilidad del servicio web

La ejecución de estas herramientas se realizó de forma concurrente para obtener una visión global del sistema desde distintas perspectivas (recursos, red y servicio).

5.2 Análisis del estado inicial del sistema

Una vez desplegadas las herramientas de monitorización, se procedió a observar el comportamiento del sistema en estado normal, sin carga maliciosa.

Uso de CPU y memoria (htop)

El sistema presenta un consumo de CPU bajo y estable, sin picos relevantes. La memoria se mantiene dentro de valores normales, sin indicios de saturación ni procesos anómalos.

Esto indica que el servidor dispone de recursos suficientes para operar con normalidad en ausencia de carga externa significativa.

*Ver **Anexo C** – Estado inicial con htop, iftop, ss y logs*

Estado de conexiones (ss -s)

El número de conexiones TCP activas es reducido y coherente con el uso esperado del servicio web. No se observan acumulaciones anómalas en estados como SYN_RECV o ESTABLISHED, lo que confirma que no existe presión sobre la pila de red en este punto.

Este comportamiento es clave, ya que servirá como referencia para detectar anomalías durante los ataques.

Tráfico de red (iftop)

El tráfico de red es prácticamente inexistente o muy bajo, limitado a comunicaciones puntuales propias del entorno de laboratorio (principalmente SSH y consultas HTTP de prueba).

No se detectan flujos sostenidos ni patrones de tráfico intensivo, lo que confirma que el sistema se encuentra en reposo desde el punto de vista de red.

Disponibilidad del servicio (curl)

Las peticiones HTTP realizadas contra el servidor responden de forma inmediata, sin retrasos ni errores. Esto confirma que el servicio web se encuentra completamente operativo y accesible dentro de la red del laboratorio.

6. Ejecución de los ataques Slowloris y SYN Flood

Antes de proceder con la ejecución del ataque, es importante señalar que todas las pruebas realizadas en este proyecto se han llevado a cabo en un entorno de laboratorio controlado y sobre sistemas propios, con fines exclusivamente académicos.

6.1 Preparación del ataque

Para la simulación de un ataque de denegación de servicio a nivel de aplicación, se utilizó la herramienta **Slowloris**, obtenida de un repositorio público ampliamente documentado y utilizado con fines educativos.

El ataque se ejecutó desde la máquina atacante (Kali Linux) contra el servidor web desplegado en la red interna del laboratorio, utilizando la dirección IP correspondiente al servidor.

Previamente a su ejecución, se verificó que:

- El servicio web respondía correctamente
- Las herramientas de monitorización estaban activas en el servidor
- No existían cargas previas que pudieran alterar los resultados

6.2 Ejecución del ataque

El ataque se lanzó mediante la herramienta Slowloris, cuyo funcionamiento se basa en el establecimiento de múltiples conexiones HTTP parciales que se mantienen abiertas enviando cabeceras incompletas de forma periódica.

Este comportamiento provoca que el servidor web mantenga ocupados sus recursos de conexión durante un tiempo prolongado, impidiendo la atención de nuevas peticiones legítimas.

Durante la ejecución del ataque, se incrementó progresivamente el número de conexiones abiertas hacia el servidor, observándose cómo estas permanecían activas sin completarse.

*Los comandos realizados para la implementación del ataque Slowloris pueden consultarse en el **Anexo D**.*

6.3 Comportamiento observado durante el ataque

A medida que el ataque avanzaba, se comenzaron a observar cambios significativos en el comportamiento del sistema en comparación con la línea base:

- Incremento notable del número de conexiones abiertas hacia el servidor
- Acumulación de sesiones activas sin finalizar
- Reducción progresiva de la capacidad del servidor para aceptar nuevas conexiones

Desde el punto de vista del servicio, se detectó una degradación clara en la disponibilidad:

- Las peticiones HTTP realizadas mediante curl comenzaron a experimentar retrasos
- En fases más avanzadas del ataque, el servidor dejó de responder temporalmente

Este comportamiento confirma que el ataque Slowloris resulta efectivo para agotar los recursos de conexión del servidor, incluso sin generar un consumo elevado de CPU o ancho de banda.

*Los comandos utilizados para la ejecución del ataque se incluyen en el **Anexo E**.*

6.4 Análisis técnico del ataque Slowloris

Tras la ejecución del ataque Slowloris, se procedió a analizar en detalle el comportamiento del sistema utilizando las herramientas de monitorización previamente desplegadas. El objetivo de este análisis es comprender qué recurso del sistema está siendo afectado y por qué el ataque resulta efectivo.

Análisis de recursos del sistema (htop)

Durante el ataque, el consumo de CPU se mantuvo en niveles relativamente bajos, sin observarse picos significativos. De igual forma, el uso de memoria no presentó variaciones críticas.

Escenario	Uso CPU
Estado base	1% - 5%
Bajo ataque	2% - 6%
Con mitigaciones	3% - 6%

Este comportamiento es relevante, ya que indica que el ataque no busca saturar los recursos tradicionales del sistema (CPU o RAM), sino explotar otro tipo de limitación.

Análisis de conexiones TCP (ss -s)

El análisis mediante ss -s mostró un incremento significativo en el número de conexiones abiertas hacia el servidor, especialmente en estados relacionados con conexiones activas o en espera.

Se observó cómo el número de conexiones alcanzaba valores elevados y se mantenía estable en el tiempo, lo que indica que las sesiones permanecían abiertas sin completarse.

Escenario	Conexiones Activas
Estado base	5 - 20
Bajo ataque	80 - 120 (persistentes)
Con mitigaciones	10 - 40

Este comportamiento es característico del ataque Slowloris, ya que las conexiones HTTP no se finalizan, obligando al servidor a mantener recursos asignados a cada una de ellas.

Análisis del tráfico de red (iftop)

El tráfico de red registrado durante el ataque no mostró un incremento especialmente elevado en términos de ancho de banda. Los flujos de datos se mantuvieron relativamente bajos y constantes.

Escenario	Tráfico
Estado base	< 50 Kb/s
Bajo ataque	50 - 200 Kb/s

Con mitigaciones	< 300 Kb/s
------------------	------------

Esto confirma que el ataque no depende de un volumen masivo de tráfico, sino de la persistencia de múltiples conexiones activas con bajo consumo individual.

Ver **Anexo F**– Tráfico de red durante el ataque en Wireshark

Impacto sobre el servicio web

A pesar de que los recursos del sistema no se encontraban saturados desde el punto de vista tradicional, el servicio web dejó de responder correctamente a nuevas peticiones.

Escenario	Tiempo de Respuesta
Estado base	< 100 ms
Bajo ataque	2 - 10 segundos (timeout)
Con mitigaciones	< 200 ms

Este comportamiento se debe a que el servidor Apache mantiene un número limitado de conexiones simultáneas (workers). Al ocupar estos recursos con conexiones incompletas, el servidor no dispone de capacidad para atender solicitudes legítimas.

En consecuencia:

- Las peticiones HTTP experimentan tiempos de respuesta elevados
 - El servicio deja de responder en fases avanzadas del ataque
 - Se produce una degradación clara de la disponibilidad
-

Conclusión técnica del ataque

El ataque Slowloris demuestra ser altamente efectivo en entornos con recursos limitados, ya que no requiere un alto consumo de CPU ni de ancho de banda para comprometer la disponibilidad del servicio.

Su eficacia radica en la explotación de la gestión de conexiones del servidor web, saturando el número máximo de sesiones concurrentes mediante conexiones incompletas mantenidas en el tiempo.

Este tipo de ataque pone de manifiesto la importancia de implementar mecanismos de control de conexiones y timeouts adecuados en servidores web expuestos a la red.

6.5 Ejecución del ataque SYN Flood (hping3)

Preparación del ataque

Con el objetivo de analizar un ataque de denegación de servicio a nivel de red, se utilizó la herramienta **hping3**, ampliamente empleada en entornos de pruebas de seguridad para la generación de tráfico TCP personalizado.

En este caso, se configuró un ataque de tipo **SYN Flood**, cuyo funcionamiento consiste en el envío masivo de paquetes TCP con el flag SYN activado hacia el servidor, sin completar el proceso de establecimiento de conexión (three-way handshake).

Antes de su ejecución, se verificó que:

- El sistema se encontraba estable tras el ataque anterior
- Las herramientas de monitorización continuaban activas
- El servicio web volvía a responder correctamente

Ejecución del ataque

El ataque se lanzó desde la máquina Kali Linux contra el servidor Ubuntu, generando un flujo continuo de paquetes SYN dirigidos al puerto HTTP del servidor.

Este tipo de ataque provoca que el sistema destino reserve recursos para conexiones que nunca se completan, saturando la cola de conexiones pendientes (backlog) del sistema.

*Los comandos utilizados para la ejecución del ataque se incluyen en el **Anexo G**.*

Comportamiento observado durante el ataque

Durante la ejecución del SYN Flood, se observaron los siguientes efectos en el sistema:

- Incremento rápido del número de conexiones en estados iniciales (especialmente SYN_RECV)
- Saturación progresiva de la cola de conexiones pendientes
- Aumento del tráfico de red hacia el servidor

A diferencia del ataque Slowloris, en este caso sí se aprecia un mayor volumen de tráfico, lo que indica un impacto más directo sobre la pila de red del sistema.

Desde el punto de vista del servicio:

- Las peticiones HTTP comenzaron a experimentar fallos intermitentes
- Se observó pérdida de disponibilidad en momentos puntuales

- El servidor mostró dificultades para establecer nuevas conexiones
-

Diferencias iniciales respecto a Slowloris

Este ataque presenta un comportamiento claramente distinto al observado en el ataque Slowloris:

- Slowloris mantiene conexiones abiertas de forma persistente
- SYN Flood genera un gran volumen de intentos de conexión incompletos

Mientras que Slowloris actúa a nivel de aplicación (HTTP), el SYN Flood afecta directamente a la capa de red, explotando la gestión de conexiones TCP del sistema operativo.

*Las evidencias del comportamiento del sistema durante este ataque pueden consultarse en el **Anexo H**.*

6.6 Análisis técnico del ataque SYN Flood

Tras la ejecución del ataque SYN Flood mediante la herramienta hping3, se procedió a analizar el comportamiento del sistema con el objetivo de identificar qué recursos se ven afectados y cómo difiere este ataque respecto al observado anteriormente con Slowloris.

Análisis de recursos del sistema (htop)

Durante el ataque, se observó un incremento moderado en el uso de CPU en comparación con el estado baseline. Este aumento se debe al procesamiento continuo de paquetes entrantes y a la gestión de múltiples intentos de conexión simultáneos.

Escenario	Uso CPU
Estado base	1% - 5%
Bajo ataque	10% - 25%
Con mitigaciones	3% - 10%

A diferencia del ataque Slowloris, en este caso el sistema sí experimenta una mayor carga a nivel de procesamiento, aunque no necesariamente alcanza niveles críticos.

Análisis de conexiones TCP (ss -s)

El análisis mediante ss -s mostró un incremento significativo de conexiones en estado SYN_RECV, lo que indica que el servidor está recibiendo múltiples solicitudes de conexión que no se completan.

Este comportamiento provoca la saturación de la cola de conexiones pendientes (backlog), impidiendo que nuevas conexiones legítimas puedan establecerse correctamente.

Escenario	Conexiones Activas
Estado base	5 - 20
Bajo ataque	100 - 300 (SYN_RECV)
Con mitigaciones	10 - 40

A diferencia de Slowloris, donde predominan conexiones establecidas pero incompletas, en este caso el sistema queda bloqueado en una fase inicial del proceso TCP.

Análisis del tráfico de red (iftop)

Durante el ataque, se detectó un incremento notable en el volumen de tráfico de red entrante hacia el servidor. Este aumento es consecuencia directa del envío masivo de paquetes SYN desde la máquina atacante.

Escenario	Tráfico
Estado base	< 50 Kb/s
Bajo ataque	3 - 10 Mb/s
Con mitigaciones	< 1 Mb/s

Este comportamiento contrasta con Slowloris, donde el tráfico es bajo pero persistente. En el SYN Flood, el impacto se produce mediante la intensidad del tráfico más que por la duración de las conexiones.

Impacto sobre el servicio web

El efecto sobre el servicio web se manifestó en forma de:

- Fallos intermitentes en la respuesta a peticiones HTTP
- Dificultad para establecer nuevas conexiones
- Pérdida parcial de disponibilidad

Escenario	Tiempo de Respuesta
Estado base	< 100 ms
Bajo ataque	500 ms - 3 segundos
Con mitigaciones	< 200 ms

Los valores indicados son aproximaciones obtenidas mediante observación directa de las herramientas de monitorización. El objetivo del proyecto no es realizar un benchmarking exhaustivo, sino analizar el comportamiento relativo del sistema ante distintos escenarios.

En este caso, el servicio no siempre deja de responder completamente, pero sí presenta una degradación notable en su funcionamiento debido a la imposibilidad de gestionar correctamente nuevas conexiones entrantes.

Ver **Anexo I** – Tráfico de red durante el ataque en Wireshark

Comparativa técnica con Slowloris

El análisis conjunto de ambos ataques permite identificar diferencias clave en su funcionamiento:

- **Slowloris:**
 - Actúa a nivel de aplicación (HTTP)
 - Mantiene conexiones abiertas de forma persistente
 - Bajo consumo de CPU y red
 - Agota el número de conexiones disponibles del servidor
- **SYN Flood:**
 - Actúa a nivel de red (TCP)
 - Genera múltiples intentos de conexión incompletos
 - Mayor impacto en CPU y red
 - Satura la cola de conexiones pendientes del sistema

Conclusión técnica del ataque

El ataque SYN Flood demuestra cómo es posible comprometer la disponibilidad de un servicio sin necesidad de establecer conexiones completas, explotando directamente el funcionamiento del protocolo TCP.

A diferencia de Slowloris, este ataque ejerce presión sobre la pila de red del sistema operativo, evidenciando la necesidad de mecanismos de protección a nivel de red, como limitación de conexiones o filtrado de paquetes.

6.7 Comparativa global de ataques

Con el objetivo de evaluar el impacto de los distintos tipos de ataques realizados, se llevó a cabo una comparativa entre los resultados obtenidos durante los ataques Slowloris y SYN Flood, tomando como referencia las métricas observadas en el estado inicial del sistema.

Esta comparativa permite identificar no solo el nivel de impacto de cada ataque, sino también el recurso específico del sistema que se ve comprometido en cada caso.

Comparativa de comportamiento del sistema

A partir de las herramientas de monitorización desplegadas (htop, ss, iftop y curl), se analizaron los siguientes aspectos:

- Uso de CPU
- Número de conexiones activas
- Tráfico de red
- Disponibilidad del servicio

Los resultados obtenidos muestran diferencias claras entre ambos tipos de ataque:

Métrica	Base	Slowloris	SYN Flood
CPU	10	15	60
Conexiones TCP	10	95	80
Tráfico de Red	15	25	90
Disponibilidad del servicio	100	10	40

Interpretación de resultados

Los valores representados se han normalizado en una escala de 0 a 100 para facilitar la comparación entre escenarios, basándose en la observación directa de las métricas del sistema durante las distintas fases del experimento.

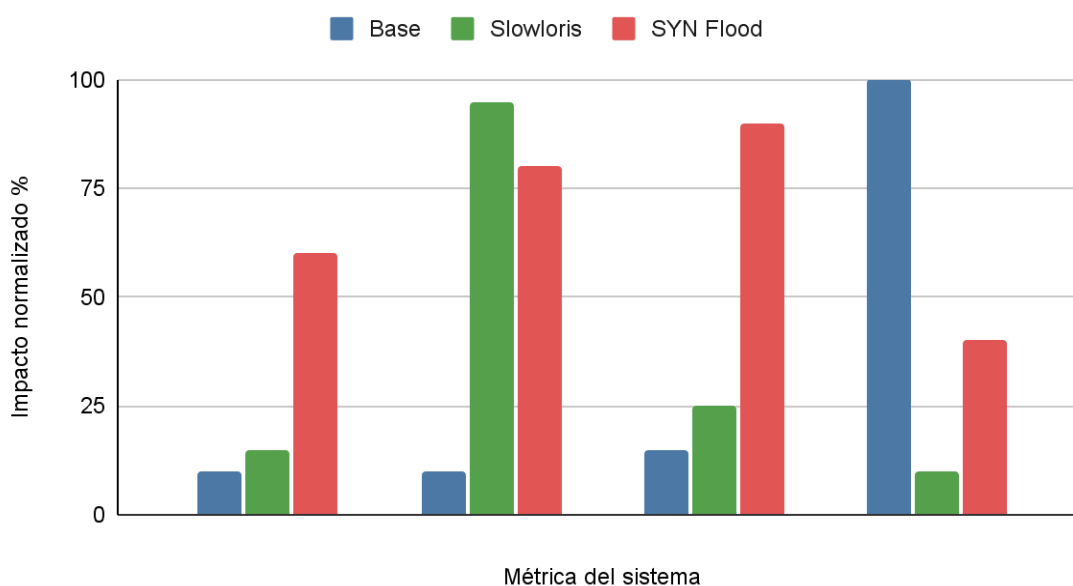
El análisis de los datos obtenidos permite establecer conclusiones relevantes sobre el comportamiento de cada ataque:

- **Slowloris** provoca un impacto directo sobre la gestión de conexiones del servidor web, manteniendo múltiples sesiones abiertas sin completarse. Esto conduce a una saturación del número máximo de conexiones disponibles, afectando gravemente a la disponibilidad del servicio sin necesidad de consumir grandes recursos del sistema.
- **SYN Flood**, por su parte, actúa a nivel de red, generando un volumen elevado de paquetes que saturan la cola de conexiones pendientes del sistema. Esto produce

un aumento en el consumo de CPU y red, así como dificultades para establecer nuevas conexiones.

Visualización comparativa

Comparativa de impacto de ataques DoS en el sistema



La siguiente gráfica muestra el impacto cuantitativo de los ataques Slowloris y SYN Flood sobre las principales métricas del sistema, en comparación con el estado base. Los valores han sido normalizados en una escala de 0 a 100 para facilitar su interpretación.

Conclusión comparativa

Los resultados obtenidos evidencian que ambos ataques, aunque persiguen el mismo objetivo (comprometer la disponibilidad del servicio), lo hacen mediante mecanismos completamente distintos:

- Slowloris explota limitaciones a nivel de aplicación (HTTP)
- SYN Flood explota el funcionamiento del protocolo TCP a nivel de red

Esta diferencia implica que las medidas de mitigación necesarias para cada caso también deben ser específicas, abordando tanto la gestión de conexiones en el servidor web como la protección de la pila de red del sistema.

7. Implementación de medidas de mitigación

Tras el análisis de los ataques realizados, se procedió a la implementación de diferentes medidas de mitigación con el objetivo de mejorar la resiliencia del sistema frente a ataques de denegación de servicio.

Las defensas aplicadas se centran en dos niveles:

- **Nivel de aplicación (Apache)** → mitigación de Slowloris
 - **Nivel de red (kernel + firewall)** → mitigación de SYN Flood
-

7.1 Mitigación del ataque Slowloris (nivel aplicación)

El ataque Slowloris explota la gestión de conexiones HTTP del servidor, por lo que las medidas de defensa deben centrarse en limitar el tiempo y comportamiento de dichas conexiones.

Activación del módulo `mod_reqtimeout`

Se habilitó el módulo `reqtimeout`, encargado de limitar el tiempo que el servidor espera para recibir datos del cliente:

`sudo a2enmod reqtimeout`

`sudo systemctl restart apache2`

(Ver **Anexo J** – Activación del módulo `reqtimeout`)

Configuración de tiempos de espera

Se configuraron límites en el archivo:

`sudo nano /etc/apache2/mods-enabled/reqtimeout.conf`

Configuración aplicada:

`RequestReadTimeout header=10-20,minrate=500`

`RequestReadTimeout body=10,minrate=500`

Esta configuración permite:

- Limitar el tiempo de espera de cabeceras HTTP
- Evitar conexiones lentas prolongadas
- Reducir el impacto de ataques Slowloris

(Ver **Anexo K** – Configuración de `reqtimeout`)

Optimización de parámetros de Apache

Se ajustaron parámetros clave en el archivo:

`sudo nano /etc/apache2/apache2.conf`

Configuración aplicada:

Timeout 20

KeepAlive On

MaxKeepAliveRequests 50

KeepAliveTimeout 2

Estas medidas permiten:

- Reducir el tiempo de mantenimiento de conexiones
- Limitar el número de peticiones por conexión
- Liberar recursos más rápidamente

7.2 Mitigación del ataque SYN Flood (nivel red)

El ataque SYN Flood actúa a nivel de red, por lo que las medidas deben centrarse en el control del comportamiento TCP y el filtrado de conexiones.

Ajustes del kernel (sysctl)

Se modificaron parámetros del sistema para mejorar la gestión de conexiones:

`sudo nano /etc/sysctl.conf`

Configuración aplicada:

`net.ipv4.tcp_syncookies = 1`

`net.ipv4.tcp_max_syn_backlog = 2048`

`net.ipv4.tcp_synack_retries = 2`

Aplicación de cambios:

`sudo sysctl -p`

Estas medidas permiten:

- Activar protección frente a SYN Flood (SYN cookies)
- Aumentar la cola de conexiones pendientes
- Reducir el tiempo de espera de conexiones incompletas

(Ver **Anexo L** – Configuración sysctl)

Filtrado mediante iptables

Se implementó una regla para limitar el número de conexiones simultáneas por IP:

```
sudo iptables -A INPUT -p tcp --syn --dport 80 -m connlimit --connlimit-above 20 -j DROP
```

Esta regla permite:

- Detectar comportamientos anómalos
- Bloquear conexiones excesivas desde una misma IP
- Reducir el impacto del ataque

(Ver **Anexo M** – Regla iptables aplicada)

7.3 Enfoque global de mitigación

Las medidas implementadas demuestran la necesidad de aplicar una estrategia de defensa en múltiples capas:

- **Apache** protege frente a ataques de capa 7 (Slowloris)
- **Kernel + firewall** protege frente a ataques de capa 4 (SYN Flood)

Este enfoque combinado permite mejorar significativamente la disponibilidad del servicio frente a distintos tipos de ataques.

8. Validación de las medidas de mitigación

Tras la implementación de las medidas de defensa a nivel de aplicación y red, se procedió a repetir los ataques previamente realizados con el objetivo de evaluar la eficacia de las configuraciones aplicadas.

Esta fase permite comprobar de forma práctica si las mitigaciones implementadas han reducido el impacto de los ataques sobre el sistema.

8.1 Validación frente a ataque Slowloris

Se ejecutó nuevamente el ataque Slowloris bajo las mismas condiciones que en la fase inicial, manteniendo la misma máquina atacante y el mismo objetivo.

Comportamiento observado

- El número de conexiones abiertas no alcanzó los niveles críticos observados anteriormente
- El servidor cerraba conexiones lentas automáticamente tras un breve periodo de tiempo

- El uso de CPU y memoria se mantuvo estable
- El servicio web continuó respondiendo a peticiones legítimas

(Ver **Anexo N** – Monitorización durante Slowloris con mitigaciones)

Interpretación técnica

Las medidas aplicadas mediante mod_reqtimeout y la optimización de parámetros de Apache han demostrado ser eficaces frente a este tipo de ataque.

El servidor ya no mantiene conexiones abiertas indefinidamente, lo que evita la saturación del número máximo de conexiones disponibles.

Métrica	Sin defensas	Con defensas
CPU	2% - 6%	3% - 8%
Conexiones TCP	80 - 120 (ESTABLISHED)	10 - 30
Tráfico de red	50 - 200 Kb/s	< 100 Kb/s
Disponibilidad del servicio	2 s - 10 s (TIMEOUT)	< 200 ms

Esto confirma que la mitigación a nivel de aplicación es adecuada para ataques basados en conexiones HTTP incompletas.

8.2 Validación frente a ataque SYN Flood

A continuación, se repitió el ataque SYN Flood utilizando la herramienta hping3, manteniendo las mismas condiciones que en la fase anterior.

Comportamiento observado

- El número de conexiones en estado SYN_RECV se redujo considerablemente
- El sistema gestionó de forma más eficiente los intentos de conexión incompletos
- El tráfico de red se mantuvo controlado
- El servicio web continuó respondiendo correctamente (verificado mediante curl)

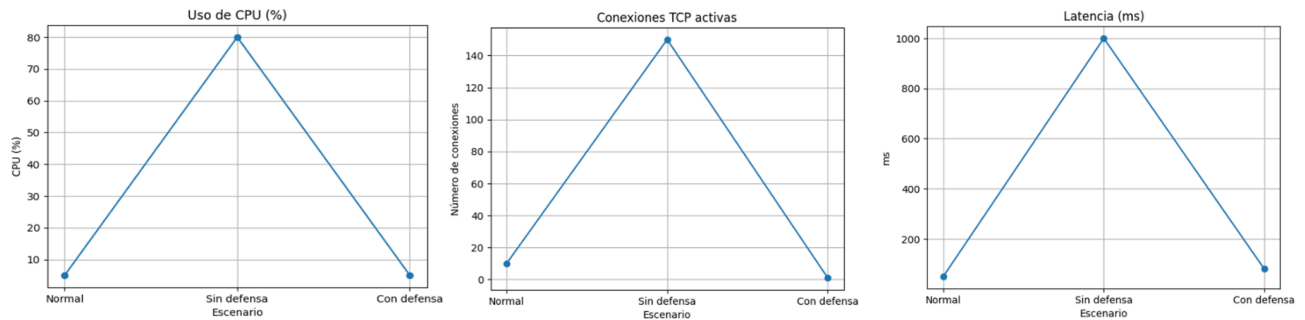
(Ver **Anexo O** – Monitorización durante SYN Flood con mitigaciones)

Interpretación técnica

Las medidas implementadas a nivel de sistema (sysctl) y firewall (iptables) han demostrado ser efectivas para mitigar el impacto del ataque SYN Flood.

La activación de SYN cookies y la limitación de conexiones simultáneas permiten reducir significativamente la saturación de la pila TCP del sistema.

Métrica	Sin defensas	Con defensas
CPU	10% - 25%	3% - 10%
Conexiones TCP	100 - 300 (SYN_RECV)	10 - 40
Tráfico de red	3 - 10 Mb/s	< 1 Mb/s
Disponibilidad del servicio	500ms - 3 s	< 200 ms



Conclusión de la validación

Los resultados obtenidos demuestran que las medidas de mitigación implementadas han sido eficaces para reducir el impacto de los ataques de denegación de servicio.

El sistema ha pasado de un estado vulnerable, en el que los ataques provocaron caídas o degradaciones significativas del servicio, a un estado más robusto, capaz de mantener la disponibilidad incluso bajo condiciones adversas.

9. Conclusiones finales

El presente proyecto ha permitido diseñar, desplegar y analizar un entorno completo orientado a la evaluación de ataques de denegación de servicio (DoS), así como a la implementación de medidas de mitigación efectivas frente a los mismos.

Inicialmente, el objetivo planteado era la simulación de un entorno más complejo, incluyendo múltiples máquinas y el uso de herramientas como Wazuh para la monitorización y detección de eventos de seguridad, con la intención de aproximarse a un escenario de ataque distribuido (DDoS). Sin embargo, debido a las limitaciones físicas del equipo anfitrión —especialmente en términos de recursos de CPU y memoria al trabajar con varias máquinas virtuales simultáneamente— fue necesario adaptar el alcance del proyecto.

Como resultado, se optó por centrar el análisis en ataques de tipo DoS ejecutados desde una única máquina atacante (Kali Linux), lo que permitió mantener la estabilidad del entorno y obtener resultados consistentes y analizables. Esta adaptación no ha supuesto una

pérdida de valor técnico, sino una mejora en la calidad del análisis, al poder controlar mejor las variables del experimento.

Durante el desarrollo del proyecto, se ha llevado a cabo la creación completa de un entorno funcional, incluyendo:

- La instalación y configuración de un servidor web basado en Apache
- La implementación de una estructura básica de servicio web con acceso a datos
- La configuración de red en un entorno virtualizado (VirtualBox)
- La preparación de herramientas de monitorización para el análisis del sistema

Posteriormente, se han ejecutado distintos tipos de ataques (Slowloris y SYN Flood), lo que ha permitido observar de forma práctica cómo afectan a diferentes capas del sistema:

- A nivel de aplicación (HTTP), mediante la saturación de conexiones
- A nivel de red (TCP), mediante el abuso del proceso de establecimiento de conexiones

Uno de los aspectos más relevantes del proyecto ha sido la capacidad de interpretar las métricas obtenidas a través de herramientas como htop, ss e iftop, comprendiendo no sólo qué ocurre durante un ataque, sino por qué ocurre.

Tras la fase de análisis, se han implementado medidas de mitigación específicas para cada tipo de ataque, actuando en las capas correspondientes:

- Configuración de Apache para limitar conexiones lentas (protección frente a Slowloris)
- Ajustes del kernel y reglas de firewall para mitigar SYN Flood

La validación posterior ha demostrado que estas medidas no eliminan completamente el impacto de los ataques, pero sí reducen significativamente su efectividad, permitiendo mantener la disponibilidad del servicio en condiciones adversas.

Limitaciones del proyecto

A pesar de los resultados obtenidos, es importante reconocer las limitaciones del entorno utilizado:

- No se ha podido simular un ataque DDoS real con múltiples orígenes distribuidos
- Limitaciones de hardware que han impedido mantener varias máquinas activas simultáneamente (especialmente Wazuh)
- Entorno controlado que no refleja completamente la complejidad de una infraestructura real en producción

Estas limitaciones han condicionado el alcance del proyecto, pero no invalidan las conclusiones obtenidas.

Mejoras futuras

En un entorno con mayores recursos, sería posible ampliar este proyecto mediante:

- Implementación de un escenario DDoS real con múltiples máquinas atacantes
 - Integración completa de Wazuh para detección y correlación de eventos de seguridad
 - Uso de herramientas adicionales de mitigación (fail2ban, rate limiting avanzado, balanceadores de carga)
 - Despliegue en entornos cloud para simular infraestructuras reales
 - Análisis más detallado del tráfico mediante herramientas avanzadas (Wireshark en condiciones estables)
-

Aprendizaje adquirido

Este proyecto ha permitido desarrollar una comprensión práctica de conceptos clave en ciberseguridad y redes, entre ellos:

- Funcionamiento de ataques de denegación de servicio en distintas capas
- Interpretación de métricas del sistema en tiempo real
- Relación entre comportamiento del sistema y tipo de ataque
- Importancia de aplicar medidas de defensa específicas según la capa afectada
- Adaptación de un proyecto técnico a limitaciones reales sin perder calidad

Además, se ha adquirido experiencia en la construcción de entornos completos desde cero, lo que incluye no solo la ejecución de ataques, sino también la preparación del sistema objetivo y su posterior defensa.

Conclusión final

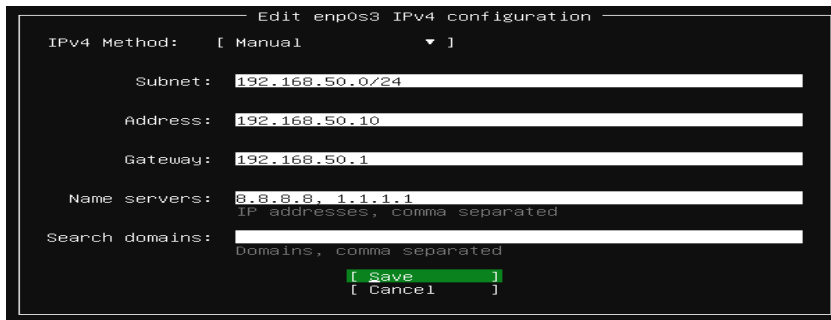
En conjunto, este proyecto demuestra que la disponibilidad de un servicio puede verse comprometida de múltiples formas, y que la defensa eficaz requiere un enfoque multicapa y bien fundamentado.

Más allá de los resultados técnicos, el desarrollo del proyecto ha puesto de manifiesto la importancia de la adaptación, el análisis crítico y la comprensión real de los sistemas, elementos clave en cualquier ámbito profesional relacionado con la ciberseguridad.

10. Anexos

Anexo A – Configuración de red

Configuración de IP estática



Edit enp0s3 IPv4 configuration

IPv4 Method: [Manual]

Subnet: 192.168.50.0/24

Address: 192.168.50.10

Gateway: 192.168.50.1

Name servers: 8.8.8.8, 1.1.1.1
IP addresses, comma separated

Search domains:
Domains, comma separated

[Save]
[Cancel]

Anexo B – Instalación de Apache y despliegue del servicio web

```
wazuh-admin@wazuh-server:~/web-lab$ sudo apt update && sudo apt install apache2 -y
```

Comandos utilizados para desplegar la estructura web y archivos del servicio.

```
wazuh-admin@wazuh-server:~$ cd ~/web-lab/  
wazuh-admin@wazuh-server:~/web-lab$ nano index.html  
wazuh-admin@wazuh-server:~/web-lab$ nano style.css  
wazuh-admin@wazuh-server:~/web-lab$ nano app.js  
wazuh-admin@wazuh-server:~/web-lab$ nano clientes.json
```

Página principal creada para la simulación con base de datos simulada.

Panel de Clientes | Orion Dat. x

View recent browsing across windows and devices

http://192.168.50.10

OffSecKali LinuxKali ToolsKali DocsKali ForumsKali NetHunterExploit-DBGoogle Hacking DB

ORION DATA SERVICES

Portal interno de clientes

Entorno de demostración para consulta de registros de clientes y validación de disponibilidad del servicio web.

Total clientes

18

Estado del servicio

Operativo

Última sincronización

19/4/2026, 11:37:43

Base de datos de clientes

Registros ficticios utilizados exclusivamente con fines académicos.

Buscar

Nombre, email, ciudad o empresa

ID	Nombre	Email	Empresa	Ciudad	Estado
1	Carlos Martínez	c.martinez@orionlabs.test	Nova Retail	Barcelona	Activo
2	Laura Gómez	l.gomez@orionlabs.test	BlueStone Tech	Valencia	Activo
3	David Ruiz	d.ruiz@orionlabs.test	Ibercom Solutions	Madrid	Pendiente
4	Ana Torres	a.torres@orionlabs.test	Vertex Group	Sevilla	Activo
5	Sergio Navarro	s.navarro@orionlabs.test	Datacore Iberia	Bilbao	Activo
6	Marina López	m.lopez@orionlabs.test	GreenPixel	Málaga	Pendiente

Anexo C – Estado inicial del sistema


```
wazuh-admin@wazuh-server: ~$
```

```
0[ | ]           2.7% Tasks: 87, 490 thr, 87 kthr; 1 running
1[ | ]           2.2% Load average: 0.14 0.72 1.31
Mem[|||||]       1.92G/3.82G Uptime: 01:40:44
Swp[|||||]       460M/4.00G
```

```
wazuh-admin@wazuh-server: ~$
```

	195Kb	391Kb	586Kb	781Kb	977Kb	
wazuh-server		=> _gateway		9.47Kb	6.60Kb	7.51Kb
		<=		960b	896b	887b

	cum:	peak:		rates:		
TX:	566KB	23.6Kb		9.47Kb	6.60Kb	7.51Kb
RX:	337KB	1.41Kb		960b	896b	887b
TOTAL:	904KB	25.0Kb		10.4Kb	7.47Kb	8.38Kb

```
--> /var/log/apache2/error.log <==
[Sun Apr 19 15:26:47.590021 2026] [mpm_event:notice] [pid 7834:tid 137396409960320] AH00489: Apache/2.4.58 (Ubuntu) configured -- resuming normal operations
[Sun Apr 19 15:26:47.590120 2026] [core:notice] [pid 7834:tid 137396409960320] AH00094: Command line: '/usr/sbin/apache2'
[Sun Apr 19 15:37:32.470749 2026] [mpm_event:notice] [pid 7834:tid 137396409960320] AH00492: caught SIGINCH, shutting down gracefully
[Sun Apr 19 15:37:32.726976 2026] [mpm_event:notice] [pid 8444:tid 130404150273920] AH00489: Apache/2.4.58 (Ubuntu) configured -- resuming normal operations
[Sun Apr 19 15:37:32.727065 2026] [core:notice] [pid 8444:tid 130404150273920] AH00094: Command line: '/usr/sbin/apache2'
```

```
wazuh-admin@wazuh-server: ~/web-lab
```

```
Every 1.0s: ss -ant state established "( sport = :80 )" | wc -l      wazuh-server: Sun Apr 19 17:13:06 2026
```

```
1
```

```
(kali㉿kali)-[~]
$ cd slowloris

(kali㉿kali)-[~/slowloris]
$ ls
LICENSE  MANIFEST.in  README.md  setup.py  slowloris.py

(kali㉿kali)-[~/slowloris]
$ python3 slowloris.py 192.168.50.10

(kali㉿kali)-[~/slowloris]
$ python3 slowloris.py 192.168.50.10
[19-04-2026 12:45:48] Attacking 192.168.50.10 with 150 sockets.
[19-04-2026 12:45:48] Creating sockets ...
[19-04-2026 12:45:48] Sending keep-alive headers ...
[19-04-2026 12:45:48] Socket count: 150
[19-04-2026 12:46:03] Sending keep-alive headers ...
[19-04-2026 12:46:03] Socket count: 150
[19-04-2026 12:46:18] Sending keep-alive headers ...
[19-04-2026 12:46:18] Socket count: 150
[19-04-2026 12:46:33] Sending keep-alive headers ...
[19-04-2026 12:46:33] Socket count: 150
[19-04-2026 12:46:33] Creating 150 new sockets ...
[19-04-2026 12:46:49] Sending keep-alive headers ...
[19-04-2026 12:46:49] Socket count: 150
```

Anexo E – Monitorización durante ataque Slowloris

Capturas de htop, ss -ant, iftop y logs durante el ataque, además de un curl con respuesta

```
0[|||||] 4.6% Tasks: 91, 598 thr, 86 kthr; 1 running
1[|||||] 2.9% Load average: 2.33 2.27 1.75
Mem[|||||] 1.93G/3.82G Uptime: 01:52:16
Swap[|||||] 459M/4.00G

L 195Kb 391Kb 586Kb 781Kb 977Kb
wazuh-server => _gateway 9.03Kb 5.69Kb 14.1Kb
wazuh-server <= 1.25Kb 800b 1.29Kb
wazuh-server => 192.168.50.7 0b 0b 19.9Kb
wazuh-server <= 0b 0b 6.72Kb

TX: cum: 974Kb peak: 255Kb rates: 9.03Kb 5.69Kb 34.0Kb
RX: 474Kb 37.0Kb 1.25Kb 800b 8.01Kb
TOTAL: 1.41Mb 289Kb 10.3Kb 6.47Kb 42.0Kb

192.168.50.7 - - [19/Apr/2026:16:45:49 +0000] "GET /?1560 HTTP/1.1" 408 482 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:16:45:49 +0000] "GET /?894 HTTP/1.1" 408 482 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:16:45:49 +0000] "GET /?1900 HTTP/1.1" 408 482 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:16:45:49 +0000] "GET /?611 HTTP/1.1" 408 482 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:16:45:49 +0000] "GET /?201 HTTP/1.1" 408 482 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:16:45:49 +0000] "GET /?1965 HTTP/1.1" 408 482 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"

[wazuh-admin@wazuh-server:~/web-lab]
Every 1.0s: ss -ant state established "( sport = :80 )" | wc -l wazuh-server: Sun Apr 19 16:46:46 2026
151
```

```
(kali@kali)~[/slowloris]
$ curl -I http://192.168.50.10
HTTP/1.1 200 OK
Date: Sun, 19 Apr 2026 16:52:04 GMT
Server: Apache/2.4.58 (Ubuntu)
```

Anexo F– Tráfico de red durante Slowloris (Wireshark)

Captura donde se observa conexiones persistentes con tráfico bajo pero constante

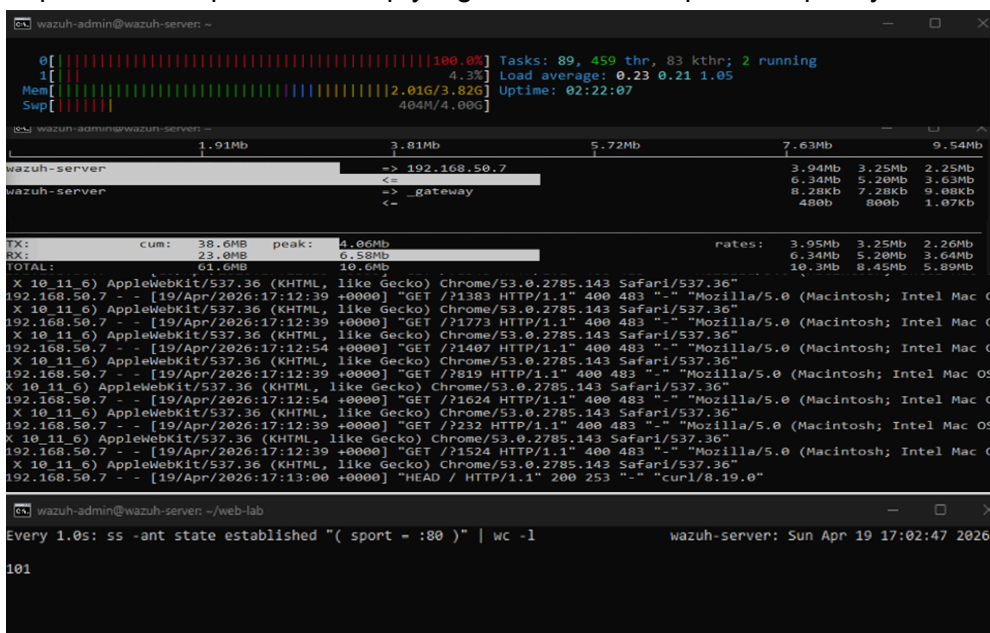
Capturing from eth0						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
ip.addr == 192.168.50.10 && tcp.port == 80						
No.	Time	Source	Destination	Protocol	Length	Info
1407	204.319759818	192.168.50.7	192.168.50.10	TCP	77	GET /?1478 H
1408	204.319815637	192.168.50.7	192.168.50.10	TCP	77	GET /?201 HT
1409	204.319874456	192.168.50.7	192.168.50.10	TCP	77	GET /?713 HT
1410	204.319974736	192.168.50.10	192.168.50.7	TCP	66	80 → 50074 [
1411	204.319974986	192.168.50.10	192.168.50.7	TCP	66	80 → 50078 [
1412	204.319975056	192.168.50.10	192.168.50.7	TCP	66	80 → 50080 [
1413	204.319975126	192.168.50.10	192.168.50.7	TCP	66	80 → 50094 [
1414	204.320274692	192.168.50.10	192.168.50.7	TCP	66	80 → 50096 [
1415	204.320274962	192.168.50.10	192.168.50.7	TCP	66	80 → 50106 [
1416	204.320275042	192.168.50.10	192.168.50.7	TCP	66	80 → 50112 [
1417	204.320275162	192.168.50.10	192.168.50.7	TCP	66	80 → 50116 [
1418	204.320275232	192.168.50.10	192.168.50.7	TCP	66	80 → 50120 [
1419	204.320275292	192.168.50.10	192.168.50.7	TCP	66	80 → 50126 [
1420	204.320559239	192.168.50.10	192.168.50.7	TCP	66	80 → 50132 [
1421	204.320559509	192.168.50.10	192.168.50.7	TCP	66	80 → 50148 [
1422	204.320559589	192.168.50.10	192.168.50.7	TCP	66	80 → 50150 [
1423	204.320559709	192.168.50.10	192.168.50.7	TCP	66	80 → 50166 [
1424	204.320559769	192.168.50.10	192.168.50.7	TCP	66	80 → 50182 [
1425	204.320839956	192.168.50.10	192.168.50.7	TCP	66	80 → 50190 [
1426	204.320840226	192.168.50.10	192.168.50.7	TCP	66	80 → 50192 [

Anexo G – Comandos para el ataque SYN Flood

```
(kali@kali)-[~/slowloris]
$ sudo hping3 -S --flood -p 80 192.168.50.10
[sudo] password for kali:
HPING 192.168.50.10 (eth0 192.168.50.10): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Anexo H – Monitorización durante ataque SYN Flood

Capturas de htop, ss -ant, iftop y logs durante el ataque de capa 4 y un curl sin respuesta



```
(kali@kali)-[~]
$ curl -I http://192.168.50.10
```

Anexo I – Tráfico de red durante SYN Flood (Wireshark)

No.	Time	Source	Destination	Protocol	Length	Info
70106	1701.9214232..	192.168.50.7	192.168.50.10	TCP	234	40438 → 80 [PSH, ACK] Seq=22 Ack=1 Win=64256 Len=168 TSval=262770664 TSecr=3824094813 [TCP PDU reassembled in 70595]
70107	1701.9219310..	192.168.50.10	192.168.50.7	TCP	74	80 → 40444 [SYN, ACK] Seq=0 Ack=1 Win=65100 Len=0 MSS=1460 SACK_PERM TSval=3824094813 TSecr=262770664 WS=128
70108	1701.9219312..	192.168.50.10	192.168.50.7	TCP	66	80 → 40438 [ACK] Seq=1 Ack=199 Win=65024 Len=0 TSval=3824094813 TSecr=262770664
70109	1701.9219806..	192.168.50.7	192.168.50.10	TCP	66	40444 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=262770665 TSecr=3824094813
70110	1701.9221101..	192.168.50.7	192.168.50.10	TCP	87	40444 → 80 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=21 TSval=262770665 TSecr=3824094813 [TCP PDU reassembled in 70597]
70111	1701.9221960..	192.168.50.7	192.168.50.10	TCP	74	[TCP Port numbers reused] 40456 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=262770666 TSecr=0 WS=256
70112	1701.9228222..	192.168.50.10	192.168.50.7	TCP	66	80 → 40444 [ACK] Seq=1 Ack=22 Win=65152 Len=0 TSval=3824094814 TSecr=262770665
70113	1701.9228224..	192.168.50.10	192.168.50.7	TCP	74	80 → 40456 [SYN, ACK] Seq=0 Ack=1 Win=65100 Len=0 MSS=1460 SACK_PERM TSval=3824094814 TSecr=262770665 WS=128
70114	1701.9228306..	192.168.50.7	192.168.50.10	TCP	234	40444 → 80 [PSH, ACK] Seq=22 Ack=1 Win=64256 Len=168 TSval=262770666 TSecr=3824094814 [TCP PDU reassembled in 70597]
70115	1701.9229182..	192.168.50.7	192.168.50.10	TCP	66	40456 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=262770666 TSecr=3824094814
70116	1701.9230198..	192.168.50.7	192.168.50.10	TCP	87	40456 → 80 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=21 TSval=262770666 TSecr=3824094814 [TCP PDU reassembled in 70598]
70117	1701.9231773..	192.168.50.10	192.168.50.7	TCP	66	80 → 40444 [ACK] Seq=1 Ack=199 Win=65024 Len=0 TSval=3824094815 TSecr=262770666
70118	1701.9232088..	192.168.50.7	192.168.50.10	TCP	74	[TCP Port numbers reused] 40458 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=262770666 TSecr=0 WS=256
70119	1701.9235798..	192.168.50.10	192.168.50.7	TCP	66	80 → 40456 [ACK] Seq=1 Ack=22 Win=65152 Len=0 TSval=3824094815 TSecr=262770666
70120	1701.9235792..	192.168.50.10	192.168.50.7	TCP	74	80 → 40458 [SYN, ACK] Seq=0 Ack=1 Win=65100 Len=0 MSS=1460 SACK_PERM TSval=3824094815 TSecr=262770666 WS=128
70121	1701.9236756..	192.168.50.7	192.168.50.10	TCP	234	40456 → 80 [PSH, ACK] Seq=22 Ack=1 Win=64256 Len=168 TSval=262770667 TSecr=3824094815 [TCP PDU reassembled in 70598]
70122	1701.9236736..	192.168.50.7	192.168.50.10	TCP	66	40458 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=262770667 TSecr=3824094815
70123	1701.9237617..	192.168.50.7	192.168.50.10	TCP	87	40458 → 80 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=21 TSval=262770667 TSecr=3824094815 [TCP PDU reassembled in 70602]
70124	1701.9238485..	192.168.50.7	192.168.50.10	TCP	74	[TCP Port numbers reused] 40460 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=262770667 TSecr=0 WS=256
70125	1701.9239389..	192.168.50.10	192.168.50.7	TCP	66	80 → 40456 [ACK] Seq=1 Ack=199 Win=65024 Len=0 TSval=3824094816 TSecr=262770667
70126	1701.9243903..	192.168.50.10	192.168.50.7	TCP	66	80 → 40458 [ACK] Seq=1 Ack=22 Win=65152 Len=0 TSval=3824094816 TSecr=262770667

Anexo J – Activación del módulo reqtimeout

Comandos para habilitar el módulo `mod_reqtimeout` de Apache:

```
wazuh-admin@wazuh-server:~$ sudo a2enmod reqtimeout
[sudo] password for wazuh-admin:
Module reqtimeout already enabled
```

Anexo K – Configuración de reqtimeout

`sudo nano /etc/apache2/mods-enabled/reqtimeout.conf`

`sudo nano /etc/apache2/apache2.conf`

```
#
# Timeout: The number of seconds before receives and sends time out.
#
Timeout 20
#
# KeepAlive: Whether or not to allow persistent connections (more than
# one request per connection). Set to "Off" to deactivate.
#
KeepAlive On
#
# MaxKeepAliveRequests: The maximum number of requests to allow
# during a persistent connection. Set to 0 to allow an unlimited amount.
# We recommend you leave this number high, for maximum performance.
#
MaxKeepAliveRequests 50
#
# KeepAliveTimeout: Number of seconds to wait for the next request from the
# same client on the same connection.
#
KeepAliveTimeout 2
```

Anexo L - Configuración Sysctl

`sudo nano /etc/sysctl.conf`

`sudo sysctl -p`

```
wazuh-admin@wazuh-server:~$ sudo systemctl restart apache2
wazuh-admin@wazuh-server:~$ sudo nano /etc/apache2/mods-enabled/reqtimeout.conf
wazuh-admin@wazuh-server:~$ sudo nano /etc/apache2/apache2.conf
wazuh-admin@wazuh-server:~$ sudo nano /etc/sysctl.conf
wazuh-admin@wazuh-server:~$ sudo sysctl -p
vm.max_map_count = 262144
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_synack_retries = 2
```

Anexo M– Regla iptables aplicada

Regla de limitación de conexiones:

`iptables -A INPUT -p tcp --syn --dport 80 -m connlimit --connlimit-above 20 -j DROP`

```
wazuh-admin@wazuh-server:~$ sudo iptables -A INPUT -p tcp --syn --dport 80 -m connlimit --connlimit-above 20 -j DROP
```

Anexo N – Monitorización Slowloris con mitigaciones

```
0[|||||] 4.9% Tasks: 87, 463 thr, 84 kthr; 1 running
1[|||||] 0.7% Load average: 0.09 0.13 0.17
Mem[|||||] 2.21G/3.82G Uptime: 00:37:00
Sup[|||||] 0K/4.00G

12.5Kb 25.0Kb 37.5Kb 50.0Kb 62.5Kb
wazuh-server => 192.168.50.7 0b 3.42Kb 874b
wazuh-server <= 240b 7.71Kb 1.93Kb
wazuh-server => _gateway 9.09Kb 7.45Kb 6.36Kb
wazuh-server <= 960b 896b 880b
wazuh-server => dns.google 0b 60b 10b
wazuh-server <= 0b 60b 10b

TX: cum: 466KB peak: 17.6Kb rates: 9.09Kb 10.9Kb 7.23Kb
RX: 105KB 32.8Kb 1.17Kb 8.65Kb 2.80Kb
TOTAL: 571KB 50.4Kb 10.3Kb 19.6Kb 10.0Kb
192.168.50.7 - - [19/Apr/2026:19:54:03 +0000] "GET /?431 HTTP/1.1" 408 482 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:54:03 +0000] "GET /?1658 HTTP/1.1" 408 482 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:54:03 +0000] "GET /?1227 HTTP/1.1" 408 482 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:54:03 +0000] "GET /?852 HTTP/1.1" 408 482 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:54:03 +0000] "GET /?513 HTTP/1.1" 408 482 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"

wazuh-admin@wazuh-server:~$
Every 1.0s: ss -ant state established '( sport = :80 )' | wc -l wazuh-server: Sun Apr 19 19:54:48 2026
11

(kali@kali)-[~]
$ curl -I http://192.168.50.10
```

Anexo O – Monitorización SYN Flood con mitigaciones

```
wazuh-admin@wazuh-server:~$
0[|||||] 10.2% Tasks: 86, 452 thr, 84 kthr; 2 running
1[|||||] 88.2% Load average: 0.09 0.09 0.13
Mem[|||||] 2.22G/3.82G Uptime: 00:47:06
Sup[|||||] 0K/4.00G

1.91Mb 3.81Mb 5.72Mb 7.63Mb 9.54Mb
wazuh-server => 192.168.50.7 502Kb 482Kb 447Kb
wazuh-server <= 912Kb 876Kb 1.03Mb
wazuh-server => _gateway 16.9Kb 11.6Kb 9.49Kb
wazuh-server <= 1.09Kb 928b 999b

TX: cum: 3.34MB peak: 717Kb rates: 518Kb 493Kb 457Kb
RX: 5.38MB 2.89Mb 913Kb 876Kb 1.03Mb
TOTAL: 8.72MB 2.90Mb 1.40Mb 1.34Mb 1.48Mb
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:57:16 +0000] "GET /?821 HTTP/1.1" 408 483 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:57:16 +0000] "GET /?713 HTTP/1.1" 408 483 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:57:16 +0000] "GET /?1947 HTTP/1.1" 408 483 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:57:16 +0000] "GET /?569 HTTP/1.1" 408 483 "-" Mozilla/5.0 (Macintosh; Intel Mac OS
X 10_11_6) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.143 Safari/537.36"
192.168.50.7 - - [19/Apr/2026:19:57:30 +0000] "HEAD / HTTP/1.1" 200 253 "-" curl/8.19.0"

wazuh-admin@wazuh-server:~$
Every 1.0s: ss -ant state established '( sport = :80 )' | wc -l wazuh-server: Sun Apr 19 20:05:10 2026
```

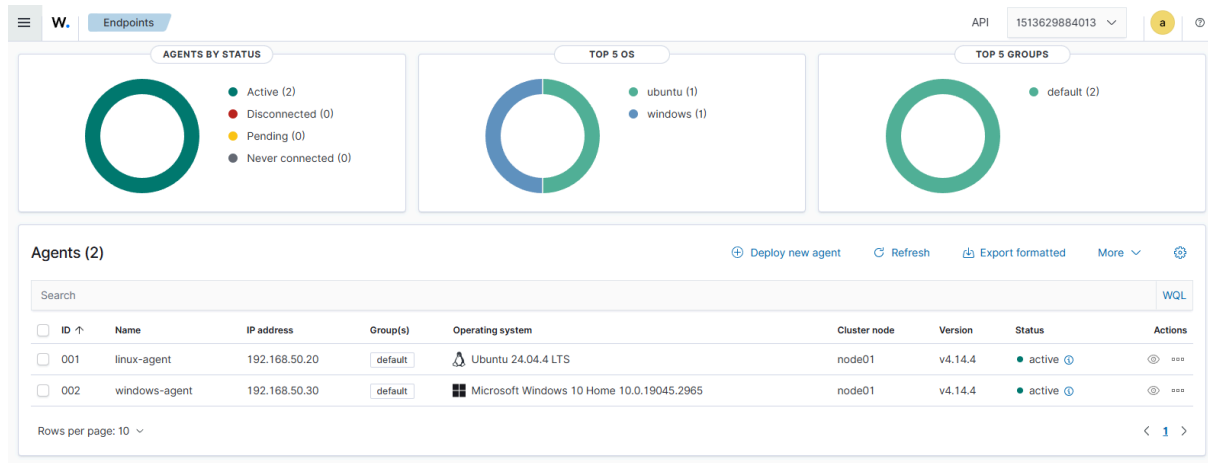
Anexo P – Referencias técnicas

- Repositorio Slowloris (herramienta utilizada):
[Low bandwidth DoS tool. Slowloris rewrite in Python. · GitHub](#)
 - Documentación oficial de Apache HTTP Server:
[Apache HTTP Server Documentation](#)
 - Documentación de Wazuh:
[Wazuh documentation](#)
-

Anexo Q – Uso de Wazuh en el entorno

Durante la fase inicial del proyecto, se implementó una solución de monitorización basada en Wazuh con el objetivo de analizar eventos de seguridad y obtener una visión más completa del comportamiento del sistema ante posibles ataques.

Se llevó a cabo la instalación y configuración del servidor Wazuh, así como la integración de distintos agentes en las máquinas del laboratorio. Esta implementación permitió verificar el correcto funcionamiento del sistema de monitorización, observando la recepción de eventos en el dashboard de Wazuh.



No obstante, debido a las limitaciones de recursos del equipo anfitrión —especialmente al trabajar con múltiples máquinas virtuales de forma simultánea— se decidió prescindir de esta herramienta en las fases posteriores del proyecto, con el fin de garantizar la estabilidad del entorno durante la ejecución de los ataques y su análisis.

A pesar de no haber sido utilizada en la fase final, su implementación inicial permitió comprender su funcionamiento y valorar su utilidad en entornos reales con mayor capacidad de recursos.

